

Jobsheet 12
Praktikum Algoritma dan Struktur Data

Nama: Dafa Naufal Rabbani
Kelas/No.Absen: TI-1G/05
NIM: 254107020086

1. Praktikum 1:

- Kode Program Mahasiswa05:
package Jobsheet12;

```
public class Mahasiswa05 {  
    String nim;  
    String nama;  
    String kelas;  
    double ipk;  
  
    public Mahasiswa05(String nim, String nama, String kelas, double  
ipk) {  
        this.nim = nim;  
        this.nama = nama;  
        this.kelas = kelas;  
        this.ipk = ipk;  
    }  
  
    public void tampil() {  
        System.out.println(  
            "NIM : " + nim +  
            "\nNama : " + nama +  
            "\nKelas : " + kelas +  
            "\nIPK : " + ipk  
        );  
    }  
}
```

- Kode Program Node05:
package Jobsheet12;

```
public class Node05 {  
    Mahasiswa05 data;  
    Node05 prev;  
    Node05 next;
```

```

    public Node05(Mahasiswa05 data) {
        this.data = data;
        this.prev = null;
        this.next = null;
    }
}

```

- Kode Program DoubleLinkedList05:


```

package Jobsheet12;

public class DoubleLinkedList05 {
    Node05 head;
    Node05 tail;

    public DoubleLinkedList05() {
        head = null;
        tail = null;
    }

    public boolean isEmpty() {
        return head == null;
    }

    public void addFirst(Mahasiswa05 data) {
        Node05 newNode = new Node05(data);
        if (isEmpty()) {
            head = tail = newNode;
        } else {
            newNode.next = head;
            head.prev = newNode;
            head = newNode;
        }
    }

    public void addLast(Mahasiswa05 data) {
        Node05 newNode = new Node05(data);
        if (isEmpty()) {
            head = tail = newNode;
        } else {
            tail.next = newNode;
            newNode.prev = tail;
            tail = newNode;
        }
    }
}

```

```

public void insertAfter(String keyNim, Mahasiswa05 data) {
    Node05 current = head;
    while (current != null && !current.data.nim.equals(keyNim)) {
        current = current.next;
    }

    if (current == null) {
        System.out.println("Data dengan NIM " + keyNim + " tidak
ditemukan.");
        return;
    }

    Node05 newNode = new Node05(data);
    if (current == tail) {
        newNode.prev = current;
        current.next = newNode;
        tail = newNode;
    } else {
        newNode.prev = current;
        newNode.next = current.next;
        current.next.prev = newNode;
        current.next = newNode;
    }
    System.out.println("Data berhasil disisipkan setelah NIM " +
keyNim);
}

public void print() {
    if (isEmpty()) {
        System.out.println("Linked list masih kosong.");
        // Bagian ini akan dimodifikasi di tugas pertanyaan
        return;
    }
    Node05 current = head;
    while (current != null) {
        current.data.tampil();
        current = current.next;
    }
}
}

```

- Kode Program DoubleLinkedListMain05:
package Jobsheet12;

```
import java.util.Scanner;
```

```
public class DoubleLinkedListMain05 {
    public static void main(String[] args) {
        Scanner scan = new Scanner(System.in);
        DoubleLinkedList05 list = new DoubleLinkedList05();
        int pilihan;

        do {
            System.out.println("\n===== MENU DOUBLE LINKED LIST
=====");
            System.out.println("1. Tambah data di awal");
            System.out.println("2. Tambah data di akhir");
            System.out.println("3. Sisipkan data di tengah (setelah NIM)");
            System.out.println("4. Hapus data di awal");
            System.out.println("5. Hapus data di akhir");
            System.out.println("6. Tampilkan data");
            System.out.println("0. Keluar");
            System.out.print("Pilih menu: ");
            pilihan = scan.nextInt();
            scan.nextLine();

            switch (pilihan) {
                case 1:
                    Mahasiswa05 mhsAwal = inputMahasiswa(scan);
                    list.addFirst(mhsAwal);
                    break;
                case 2:
                    Mahasiswa05 mhsAkhir = inputMahasiswa(scan);
                    list.addLast(mhsAkhir);
                    break;
                case 3:
                    System.out.print("Masukkan NIM yang dicari: ");
                    String keyNim = scan.nextLine();
                    System.out.println("Masukkan data baru: ");
                    Mahasiswa05 mhsBaru = inputMahasiswa(scan);
                    list.insertAfter(keyNim, mhsBaru);
                    break;
                case 4:
                    list.removeFirst();
            }
        } while (pilihan != 0);
    }
}
```

```

        break;
    case 5:
        list.removeLast();
        break;
    case 6:
        list.print();
        break;
    case 0:
        System.out.println("Program selesai.");
        break;
    default:
        System.out.println("Menu tidak valid.");
    }
} while (pilihan != 0);
scan.close();
}

```

• Hasil Running:

```

PS D:\Polinema\Semester2\Tugas\PASD\PraktikAlgoritmaDanStrukturData> &
cp' 'C:\Users\Dapaa\AppData\Roaming\Code\User\workspaceStorage\faaac69d
'Jobsheet12.DoubleLinkedListMain05'

===== MENU DOUBLE LINKED LIST =====
1. Tambah data di awal
2. Tambah data di akhir
3. Sisipkan data di tengah (setelah NIM)
4. Hapus data di awal
5. Hapus data di akhir
6. Tampilkan data
0. Keluar
Pilih menu: 2
Masukkan NIM : 123005
Masukkan Nama : Harry
Masukkan Kelas : 1A
Masukkan IPK : 3,76

===== MENU DOUBLE LINKED LIST =====
1. Tambah data di awal
2. Tambah data di akhir
3. Sisipkan data di tengah (setelah NIM)
4. Hapus data di awal
5. Hapus data di akhir
6. Tampilkan data
0. Keluar
Pilih menu: 3
Masukkan NIM yang dicari: 123005
Masukkan data baru:
Masukkan NIM : 123010
Masukkan Nama : Potter
Masukkan Kelas : 1B
Masukkan IPK : 3,55
Data berhasil disisipkan setelah NIM 123005

```

```

===== MENU DOUBLE LINKED LIST =====
1. Tambah data di awal
2. Tambah data di akhir
3. Sisipkan data di tengah (setelah NIM)
4. Hapus data di awal
5. Hapus data di akhir
6. Tampilkan data
0. Keluar
Pilih menu: 6
NIM : 123005
Nama : Harry
Kelas : 1A
IPK : 3.76
NIM : 123010
Nama : Potter
Kelas : 1B
IPK : 3.55

===== MENU DOUBLE LINKED LIST =====
1. Tambah data di awal
2. Tambah data di akhir
3. Sisipkan data di tengah (setelah NIM)
4. Hapus data di awal
5. Hapus data di akhir
6. Tampilkan data
0. Keluar
Pilih menu: █

```

- Jawaban Pertanyaan:

1. Perbedaan SLL dan DLL:

SLL hanya memiliki satu pointer (next) ke arah depan, sehingga traversal hanya searah. DLL memiliki dua pointer (prev dan next), memungkinkan traversal dua arah (maju dan mundur).

2. Fungsi next dan prev:

next menyimpan alamat node selanjutnya untuk maju, sedangkan prev menyimpan alamat node sebelumnya untuk mundur atau kembali ke node di belakangnya.

3. Fungsi Konstruktor:

Menginisialisasi head dan tail menjadi null, yang menandakan bahwa saat objek dibuat, list dalam keadaan kosong.

4. Mengapa head & tail sama saat kosong:

Karena saat penambahan pertama kali, node tersebut menjadi satu-satunya elemen, sehingga ia bertindak sebagai elemen pertama (head) sekaligus terakhir (tail).

5. Modifikasi print:

```

public void print() {
    if (isEmpty()) {
        System.out.println("Linked List masih kosong.");
        return;
    }
    Node05 current = head;
    while (current != null) {
        current.data.tampil();
    }
}

```

```

        current = current.next;
    }
}
6. Menambahkan method printReverse:
public void printReverse() {
    if (isEmpty()) {
        System.out.println("Linked List masih kosong.");
        return;
    }
    Node05 current = tail;
    while (current != null) {
        current.data.tampil();
        current = current.prev;
    }
}

```

2. Praktikum 2:

- Kode Program DoubleLinkedList05:
package Jobsheet12;

```

public class DoubleLinkedList05 {
    Node05 head;
    Node05 tail;

    public DoubleLinkedList05() {
        head = null;
        tail = null;
    }

    public boolean isEmpty() {
        return head == null;
    }

    public void addFirst(Mahasiswa05 data) {
        Node05 newNode = new Node05(data);
        if (isEmpty()) {
            head = tail = newNode;
        } else {
            newNode.next = head;
            head.prev = newNode;
            head = newNode;
        }
    }
}

```

```

    }

    public void addLast(Mahasiswa05 data) {
        Node05 newNode = new Node05(data);
        if (isEmpty()) {
            head = tail = newNode;
        } else {
            tail.next = newNode;
            newNode.prev = tail;
            tail = newNode;
        }
    }

    public void insertAfter(String keyNim, Mahasiswa05 data) {
        Node05 current = head;
        while (current != null && !current.data.nim.equals(keyNim)) {
            current = current.next;
        }

        if (current == null) {
            System.out.println("Data dengan NIM " + keyNim + " tidak
ditemukan.");
            return;
        }

        Node05 newNode = new Node05(data);
        if (current == tail) {
            newNode.prev = current;
            current.next = newNode;
            tail = newNode;
        } else {
            newNode.prev = current;
            newNode.next = current.next;
            current.next.prev = newNode;
            current.next = newNode;
        }
        System.out.println("Data berhasil disisipkan setelah NIM " +
keyNim);
    }

    public void print() {
        if (isEmpty()) {
            System.out.println("Linked List masih kosong.");
        }
    }

```



```

        return;
    }
    Node05 current = head;
    while (current != null) {
        current.data.tampil();
        current = current.next;
    }
}

public void printReverse() {
    if (isEmpty()) {
        System.out.println("Linked List masih kosong.");
        return;
    }
    Node05 current = tail;
    while (current != null) {
        current.data.tampil();
        current = current.prev;
    }
}

public void removeFirst() {
    if (isEmpty()) {
        System.out.println("Linked List kosong.");
        return;
    }
    if (head == tail) {
        head = tail = null;
    } else {
        head = head.next;
        head.prev = null;
    }
}

public void removeLast() {
    if (isEmpty()) {
        System.out.println("Linked List kosong.");
        return;
    }
    if (head == tail) {
        head = tail = null;
    } else {
        tail = tail.prev;
    }
}

```

```

        tail.next = null;
    }
}
}

```

- Hasil Running:

```

===== MENU DOUBLE LINKED LIST =====
1. Tambah data di awal
2. Tambah data di akhir
3. Sisipkan data di tengah (setelah NIM)
4. Hapus data di awal
5. Hapus data di akhir
6. Tampilkan data
0. Keluar
Pilih menu: 5
Data berhasil dihapus:
NIM   : 123010
Nama  : Potter
Kelas : 1B
IPK   : 3.55

===== MENU DOUBLE LINKED LIST =====
1. Tambah data di awal
2. Tambah data di akhir
3. Sisipkan data di tengah (setelah NIM)
4. Hapus data di awal
5. Hapus data di akhir
6. Tampilkan data
0. Keluar
Pilih menu: 6
NIM   : 123005
Nama  : Harry
Kelas : 1A
IPK   : 3.76

===== MENU DOUBLE LINKED LIST =====
1. Tambah data di awal
2. Tambah data di akhir
3. Sisipkan data di tengah (setelah NIM)
4. Hapus data di awal
5. Hapus data di akhir
6. Tampilkan data
0. Keluar
Pilih menu: 0
Program selesai.
PS D:\Polinema\Semester2\Tugas\PASD\PraktikAlgoritmaDanStrukturData>

```

Jawaban Pertanyaan:

1. Fungsi Statement:

- `head = head.next;` Memindahkan pointer head ke node kedua.
- `head.prev = null;` Memutus hubungan pointer node kedua ke node pertama yang baru dihapus agar benar-benar menjadi awal list.

2. Modifikasi Method Hapus (Menampilkan Data):

```
public void removeFirst() {  
    if (isEmpty()) {  
        System.out.println("Linked List kosong.");  
        return;  
    }  
    System.out.println("Data berhasil dihapus:");  
    head.data.tampil();  
    if (head == tail) {  
        head = tail = null;  
    } else {  
        head = head.next;  
        head.prev = null;  
    }  
}
```

```
public void removeLast() {  
    if (isEmpty()) {  
        System.out.println("Linked List kosong.");  
        return;  
    }  
    System.out.println("Data berhasil dihapus:");  
    tail.data.tampil();  
    if (head == tail) {  
        head = tail = null;  
    } else {  
        tail = tail.prev;  
        tail.next = null;  
    }  
}
```